

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN

Facultad de Ingeniería de Producción y Servicios
Escuela Profesional de Ciencia de la Computación

SEGURIDAD EN COMPUTACIÓN

Cifrados clásicos: disco de Alberti y rejilla giratoria

Práctica: implementación de cifrado y descifrado

Aplicación: elquesohelado.github.io/Alberti_Fleissner
Repositorio: github.com/ElQuesoHelado/Alberti_Fleissner

Integrantes: Alva Cornejo, Jose Javier
Carpio Paiva, Cesar Gonzalo
Palomino Rivadeneyra, Misael Jesús
Zapana Pariapaza, Gonzalo Rodrigo

Arequipa – Perú
21 de octubre de 2026

Índice

1. Objetivo y alcance **2**

2. Preprocesado y convenciones compartidas **2**

3. Cifrado y descifrado con el disco de Alberti **2**

3.1. Principio del algoritmo 2

3.2. Alfabeto, clave y limitaciones 2

3.3. Procedimiento de cifrado 3

3.4. Procedimiento de descifrado 3

3.5. Evidencias de ejecución 3

4. Cifrado y descifrado con la rejilla giratoria **4**

4.1. Principio del algoritmo 4

4.2. Alfabeto, clave y limitaciones 4

4.3. Procedimiento de cifrado 4

4.4. Procedimiento de descifrado 5

4.5. Evidencias de ejecución 5

5. Implementación y código **5**

5.1. Código fuente de los algoritmos 5

6. Conclusiones **8**

1 Objetivo y alcance

Este trabajo describe y presenta una implementación web educativa de dos cifrados clásicos: la sustitución polialfabética inspirada en el disco de Alberti y la transposición mediante rejilla giratoria de Fleissner. Para cada método se explica el principio, el alfabeto o clave, el proceso inverso de descifrado, las limitaciones de la implementación y espacios reservados para evidencias de ejecución.

La aplicación se ejecuta en el navegador. Está implementada en TypeScript con React y Vite; las funciones de cifrado y descifrado se encuentran en `src/ciphers.ts`. No requiere servidor para procesar los mensajes. Estos algoritmos se presentan con fines académicos y no deben usarse para proteger información sensible.

2 Preprocesado y convenciones compartidas

Antes de cifrar, el perfil usado en este trabajo convierte el texto a mayúsculas, elimina diacríticos (`á` pasa a `A`; `ñ` pasa a `N`), elimina espacios y saltos de línea, y filtra la entrada para conservar únicamente las letras latinas A–Z y los dígitos 0–9. Así se eliminan signos y puntuación. La aplicación permite cambiar estas opciones, pero para reproducir los resultados se debe usar el perfil descrito y compartir la configuración JSON. La vista previa permite comprobar el texto resultante.

Por ejemplo, `Ataque al amanecer!` se normaliza como `ATAQUEALAMANECER`; el texto descifrado aparece unido y en mayúsculas.

El objeto JSON que acompaña al criptograma guarda las opciones de preprocesado. Quien descifra debe recibir tanto el criptograma como esa configuración y seleccionar el algoritmo correspondiente. El texto original no se reconstruye exactamente si el preprocesado eliminó diferencias de caja, tildes o espacios; el resultado es el texto normalizado. En la rejilla, además, se guarda la longitud para retirar el relleno. En Alberti, los caracteres ajenos a los discos se conservan o eliminan según la opción seleccionada.

3 Cifrado y descifrado con el disco de Alberti

3.1 Principio del algoritmo

El método sustituye cada símbolo claro por un símbolo de un alfabeto cifrante. La implementación usa un anillo exterior como alfabeto de entrada y un alfabeto interior como alfabeto de salida. El giro inicial y un incremento periódico determinan qué posición del alfabeto interior se usa. El principio del disco móvil fue descrito por Leon Battista Alberti en *De cifris* (siglo XV) [1].

Sean P_i la posición del símbolo claro en el anillo exterior, m la longitud de los alfabetos, o el giro inicial, a el incremento y q el período. Para el símbolo cifrable número i, la posición cifrante es

$$C_i = (P_i + o + \lfloor i/q \rfloor a) \bmod m.$$

La numeración de i solo avanza al procesar un símbolo presente en el disco. El descifrado busca el símbolo en el alfabeto interior y aplica la operación inversa:

$$P_i = (C_i - o - \lfloor i/q \rfloor a) \bmod m.$$

3.2 Alfabeto, clave y limitaciones

Los alfabetos predeterminados tienen 24 símbolos y deben contener la misma cantidad de símbolos únicos:

- Exterior (claro): `ABCDEFGILMNOPQRSTVXZ1234`.
- Interior (cifrado): `usqomkhfdbacegilnprtxz&y`.

La clave de descifrado contiene ambos alfabetos, el giro inicial o, el incremento a y el período q. En la interfaz, el período debe ser al menos 1. Los alfabetos son configurables y el validador exige igual longitud y ausencia de duplicados.

La variante implementada no es una réplica completa de todas las convenciones históricas del disco. No incluye indicadores externos para anunciar cambios arbitrarios de giro. Las letras que no aparecen en el alfabeto exterior (por ejemplo, J, H, K, U, W y Ñ antes de normalizar) se consideran no soportadas: se conservan sin cifrar o se eliminan según la opción elegida. Si se conservan símbolos ajenos, esos símbolos no aportan al contador periódico. La sustitución clásica no proporciona seguridad criptográfica moderna.

3.3 Procedimiento de cifrado

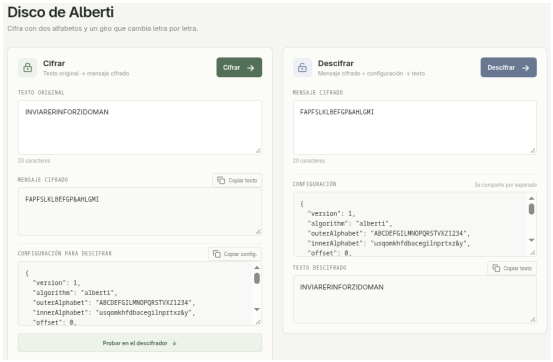
1. Aplicar las opciones de preprocesado al texto claro.
2. Validar que ambos alfabetos tengan longitud igual y símbolos únicos.
3. Para cada carácter reconocido, hallar su posición exterior, calcular el giro según o, a y q, y emitir el símbolo interior correspondiente.
4. Aplicar la política configurada a los caracteres que no estén en el alfabeto.
5. Entregar el criptograma y el JSON de parámetros.

3.4 Procedimiento de descifrado

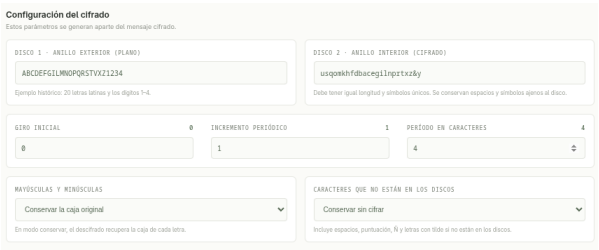
1. Recibir el criptograma y el JSON producido durante el cifrado.
2. Validar los alfabetos y recuperar giro, incremento, período y tratamiento de caracteres.
3. Contar los símbolos reconocidos en el mismo orden; buscar cada uno en el alfabeto interior y restar el giro que le corresponde módulo m.
4. Mostrar el texto claro normalizado. No se restauran tildes, espacios ni caja que hayan sido descartados antes del cifrado.

3.5 Evidencias de ejecución

La primera captura muestra el cifrado y su descifrado; la segunda muestra la configuración del disco.



Cifrado y descifrado



Configuración

Figura 1: Resultados y parámetros del disco de Alberti.

4 Cifrado y descifrado con la rejilla giratoria

4.1 Principio del algoritmo

La rejilla giratoria es un cifrado de transposición: cambia el orden de los caracteres, pero no sustituye sus valores. La configuración usada en este trabajo es una rejilla de 4×4, girada en sentido horario en cuatro posiciones. Las perforaciones se leen por filas en cada giro. Una máscara válida visita cada celda del tablero una sola vez [2], [3].

La máscara se representa con `X` para una celda perforada y `.` para una celda cerrada:

`X... / ...X / ..X. / .X..`

Durante el cifrado se escribe el texto a través de las perforaciones en cada orientación y se lee el tablero completo por filas. En el descifrado se coloca el criptograma en el mismo orden por filas y se recuperan los caracteres recorriendo las perforaciones en el mismo orden de giros. La configuración compartida contiene estas posiciones, el sentido y el número de giros, además de la longitud original del texto.

4.2 Alfabeto, clave y limitaciones

El alfabeto de entrada es `ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789`: letras latinas mayúsculas sin Ñ y dígitos. El preprocesado convierte a mayúsculas, quita tildes, convierte Ñ en N y elimina espacios, signos y cualquier otro símbolo fuera del alfabeto. El equipo de descifrado recibe la longitud original junto con el criptograma; la configuración JSON de la aplicación también registra la longitud y los parámetros de la rejilla.

La rejilla indicada tiene una perforación por cada órbita y completa 16 celdas tras cuatro giros. Si el texto ocupa menos celdas, las posiciones restantes se rellenan con `X` hasta completar el bloque. La longitud original se comunica junto con el cifrado y permite retirar el relleno al descifrar. La pérdida de caracteres o de esta longitud puede impedir la recuperación. La transposición no oculta las frecuencias de los caracteres y no ofrece seguridad moderna.

4.3 Procedimiento de cifrado

1. Aplicar el preprocesado y el filtro de caracteres seleccionados.
2. Validar la rejilla: cada órbita de rotación debe tener exactamente una perforación.

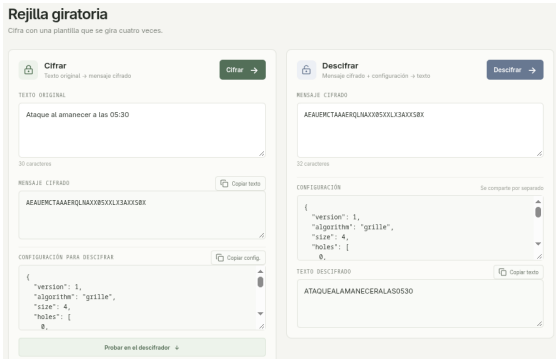
3. Recorrer por filas los huecos de cada una de las cuatro posiciones horarias y llenar las celdas.
4. Completar las celdas vacías del bloque con `X`; leer el tablero por filas y compartir la longitud original y los parámetros.

4.4 Procedimiento de descifrado

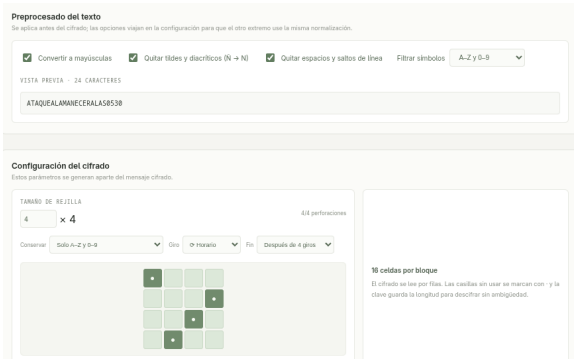
1. Cargar el criptograma junto con el JSON de configuración.
2. Validar tamaño, perforaciones y que la longitud del criptograma coincida con los bloques esperados.
3. Recorrer las mismas perforaciones y giros, extraer hasta la longitud registrada y descartar el relleno.
4. Presentar el texto filtrado y normalizado. Los espacios y símbolos que se descartaron no pueden reconstruirse.

4.5 Evidencias de ejecución

La primera captura muestra el cifrado y su descifrado; la segunda muestra el preprocesado y la configuración de la rejilla, incluida su longitud original.



Cifrado y descifrado



Preprocesado y configuración

Figura 2: Resultados y parámetros de la rejilla giratoria.

5 Implementación y código

La aplicación está escrita en TypeScript y usa React para la interfaz y Vite para el desarrollo y empaquetado web. El siguiente listado incluye las funciones implementadas de preprocesado, cifrado y descifrado de ambos métodos, tomadas directamente de `src/ciphers.ts` del proyecto [4].

5.1 Código fuente de los algoritmos

```
export const ALBERTI_OUTER = 'ABCDEFGILMNOPQRSTVXZ1234'
export const ALBERTI_INNER = 'usqomkhfdbacegilnprtxz&y'
export const LATIN = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'

export type PreprocessOptions = { uppercase: boolean; removeDiacritics: boolean; removeSpaces: boolean;
↪  keep: 'all' | 'letters' | 'alphanumeric' }
export const DEFAULT_PREPROCESS: PreprocessOptions = { uppercase: true, removeDiacritics: true,
↪  removeSpaces: true, keep: 'alphanumeric' }
```

```ts
export type AlbertiConfig = { version: 1; algorithm: 'alberti'; outerAlphabet: string; innerAlphabet: string; offset: number; advance: number; periodLength: number; unsupported: 'remove' | 'preserve'; letterCase: 'preserve' | 'uppercase'; preprocess?: PreprocessOptions }
export type GrilleConfig = { version: 1; algorithm: 'grille'; size: number; holes: number[]; length: number; direction: 'clockwise' | 'counterclockwise'; mode: 'four' | 'untilDone'; keep: 'alphanumeric' | 'nonspace'; preprocess?: PreprocessOptions }
export type CipherConfig = AlbertiConfig | GrilleConfig

export function encodeConfig(config: CipherConfig): string { return JSON.stringify(config, null, 2) }

export function preprocessText(input: string, options: PreprocessOptions): string {
  let text = input.normalize('NFC')
  if (options.removeDiacritics) text = text.normalize('NFD').replace(/\p{M}/gu, '').replace(/ñ/gi, match => match === 'Ñ' ? 'N' : 'n').normalize('NFC')
  if (options.uppercase) text = text.toLocaleUpperCase('es')
  if (options.removeSpaces) text = text.replace(/\s/gu, '')
  if (options.keep === 'letters') text = [...text].filter(ch => /[A-Za-z]/u.test(ch)).join('')
  if (options.keep === 'alphanumeric') text = [...text].filter(ch => /[A-Za-z0-9]/u.test(ch)).join('')
  return text
}

export function decodeConfig(raw: string): CipherConfig {
  const source = raw.trim().replace(/^```(?:json)?\s*/i, '').replace(/\s*```$/, '')
  const config = JSON.parse(source) as CipherConfig
  if (config.version !== 1 || !['alberti', 'grille'].includes(config.algorithm)) throw new Error('La configuración no tiene un formato válido.')
  if (config.algorithm === 'alberti') {
    config.periodLength ??= 1
    config.unsupported ??= 'preserve'; config.letterCase ??= 'preserve'
    if (!['remove', 'preserve'].includes(config.unsupported) || !['preserve', 'uppercase'].includes(config.letterCase)) throw new Error('El tratamiento de caracteres del disco no es válido.')
    validateAlberti(config.outerAlphabet, config.innerAlphabet, config.offset, config.advance, config.periodLength)
  } else {
    config.direction ??= 'clockwise'; config.mode ??= 'four'; config.keep ??= 'nonspace'
    if (!Number.isInteger(config.size) || config.size < 2 || config.size > 16 || !Array.isArray(config.holes) || !Number.isInteger(config.length) || config.length < 0 || !['clockwise', 'counterclockwise'].includes(config.direction) || !['four', 'untilDone'].includes(config.mode) || !['alphanumeric', 'nonspace'].includes(config.keep)) throw new Error('Los parámetros de la rejilla no son válidos.')
  }
  if (config.preprocess && (typeof config.preprocess.uppercase !== 'boolean' || typeof config.preprocess.removeDiacritics !== 'boolean' || typeof config.preprocess.removeSpaces !== 'boolean' || !['all', 'letters', 'alphanumeric'].includes(config.preprocess.keep))) throw new Error('Las opciones de preprocesado no son válidas.')
  return config
}

export function albertiEncrypt(input: string, outerAlphabet: string, innerAlphabet: string, offset: number, advance = 0, periodLength = 1, unsupported: 'remove' | 'preserve' = 'preserve', letterCase: 'preserve' | 'uppercase' = 'preserve'): string {
  const [outer, inner] = validateAlberti(outerAlphabet, innerAlphabet, offset, advance, periodLength)
  let position = 0
  return [...input].map(char => {
    const index = findSymbol(outer, char)
    if (index < 0) return unsupported === 'preserve' ? char : ''
    const shift = mod(offset + Math.floor(position / periodLength) * advance, outer.length)
    position++
    const result = inner[mod(index + shift, outer.length)]
    return letterCase === 'uppercase' ? result.toLocaleUpperCase() : matchCase(result, char)
  }).join('')
}

export function albertiDecrypt(input: string, outerAlphabet: string, innerAlphabet: string, offset: number, advance = 0, periodLength = 1, unsupported: 'remove' | 'preserve' = 'preserve', letterCase: 'preserve' | 'uppercase' = 'preserve'): string {
  const [outer, inner] = validateAlberti(outerAlphabet, innerAlphabet, offset, advance, periodLength)
  let position = 0
  return [...input].map(char => {
    const index = findSymbol(inner, char)
    if (index < 0) return unsupported === 'preserve' ? char : ''
    const shift = mod(offset + Math.floor(position / periodLength) * advance, outer.length)
    position++
    const result = outer[mod(index - shift, outer.length)]
    return letterCase === 'uppercase' ? result.toLocaleUpperCase() : matchCase(result, char)
```

```
  }).join('')
}

function findSymbol(alphabet: string[], char: string) {
  const exact = alphabet.indexOf(char)
  if (exact >= 0) return exact
  const upper = char.toLocaleUpperCase()
  return alphabet.findIndex(symbol => symbol.toLocaleUpperCase() === upper)
}

function matchCase(symbol: string, source: string) {
  if (source.toLocaleUpperCase() !== source.toLocaleLowerCase()) {
    if (source === source.toLocaleUpperCase()) return symbol.toLocaleUpperCase()
    if (source === source.toLocaleLowerCase()) return symbol.toLocaleLowerCase()
  }
  return symbol
}

function validateAlberti(outerText: string, innerText: string, offset: number, advance: number,
↪  periodLength: number): [string[], string[]] {
  const outer = [...outerText], inner = [...innerText]
  if (outer.length < 2 || outer.length !== inner.length || new Set(outer).size !== outer.length || new
  ↪  Set(inner).size !== inner.length || !Number.isInteger(offset) || !Number.isInteger(advance) ||
  ↪  !Number.isInteger(periodLength) || periodLength < 1) throw new Error('Los discos deben tener
  ↪  alfabetos únicos de igual longitud; período mínimo: 1.')
  return [outer, inner]
}

function mod(value: number, divisor: number) { return ((value % divisor) + divisor) % divisor }
function rotateCell(cell: number, size: number, direction: 'clockwise' | 'counterclockwise'): number {
  const row = Math.floor(cell / size), col = cell % size
  return direction === 'clockwise' ? col * size + size - 1 - row : (size - 1 - col) * size + row
}
function rotatedHoles(size: number, holes: number[], turn: number, direction: 'clockwise' |
↪  'counterclockwise') {
  return holes.map(hole => { let cell = hole; for (let i = 0; i < turn; i++) cell = rotateCell(cell, size,
  ↪  direction); return cell }).sort((a, b) => a - b)
}

export function grilleOrbits(size: number, direction: 'clockwise' | 'counterclockwise' = 'clockwise'):
↪  number[][] {
  const seen = new Set<number>(), orbits: number[][] = []
  for (let cell = 0; cell < size * size; cell++) {
    if (seen.has(cell)) continue
    let current = cell; const orbit: number[] = []
    do { orbit.push(current); seen.add(current); current = rotateCell(current, size, direction) } while
    ↪  (current !== cell)
    orbits.push(orbit)
  }
  return orbits
}

export function validGrille(size: number, holes: number[], direction: 'clockwise' | 'counterclockwise' =
↪  'clockwise') {
  const orbits = grilleOrbits(size, direction)
  return holes.length === orbits.length && new Set(holes).size === holes.length && holes.every(h =>
  ↪  Number.isInteger(h) && h >= 0 && h < size * size) && new Set(holes.map(h => orbits.findIndex(o =>
  ↪  o.includes(h)))).size === orbits.length
}

export function prepareGrilleText(input: string, keep: 'alphanumeric' | 'nonspace') {
  return [...input].filter(ch => keep === 'alphanumeric' ? /[A-Za-z0-9]/u.test(ch) : !/\s/u.test(ch))
}

export function grilleEncrypt(input: string, size: number, holes: number[], direction: 'clockwise' |
↪  'counterclockwise' = 'clockwise', mode: 'four' | 'untilDone' = 'four', keep: 'alphanumeric' |
↪  'nonspace' = 'nonspace'): { text: string; length: number } {
  if (!Number.isInteger(size) || size < 2 || size > 16 || !validGrille(size, holes, direction)) throw new
  ↪  Error('La rejilla debe tener una perforación por cada órbita de rotación.')
  const chars = prepareGrilleText(input, keep), capacity = size * size, output: string[] = []
  for (let start = 0; start < chars.length || (chars.length === 0 && start === 0); start += capacity) {
    const block = chars.slice(start, start + capacity), grid = Array<string>(capacity).fill('·'), used =
    ↪  new Set<number>()
    let cursor = 0, turns = 0
    while (turns < 4 && (mode === 'four' || cursor < block.length)) {
      for (const cell of rotatedHoles(size, holes, turns, direction)) {
```

```
        if (used.has(cell)) continue
        used.add(cell)
        grid[cell] = block[cursor++] ?? 'X'
      }
      turns++
    }
    output.push(...grid)
    if (!chars.length) break
  }
  return { text: output.join(''), length: chars.length }
}

export function grilleDecrypt(input: string, size: number, holes: number[], length: number, direction:
↪  'clockwise' | 'counterclockwise' = 'clockwise', mode: 'four' | 'untilDone' = 'four'): string {
  const chars = [...input], capacity = size * size
  if (!Number.isInteger(size) || size < 2 || size > 16 || !validGrille(size, holes, direction) ||
  ↪  !Number.isInteger(length) || length < 0 || chars.length !== Math.max(1, Math.ceil(length / capacity))
  ↪  * capacity) throw new Error('La clave o la longitud del mensaje no corresponden a esta rejilla.')
  const output: string[] = []
  for (let start = 0; start < chars.length; start += capacity) {
    const block = chars.slice(start, start + capacity), used = new Set<number>()
    let read = 0, turns = 0
    while (turns < 4 && (mode === 'four' || read < length - start)) {
      for (const cell of rotatedHoles(size, holes, turns, direction)) {
        if (used.has(cell)) continue
        used.add(cell)
        const char = block[cell]
        if (char !== '·' && read < length - start) { output.push(char); read++ }
      }
      turns++
    }
  }
  return output.slice(0, length).join('')
}
```

6 Conclusiones

Se implementaron dos principios criptográficos distintos: sustitución con alfabetos móviles para Alberti y transposición con una máscara rotatoria para Fleissner. El preprocesado y la configuración compartida permiten fijar convenciones compatibles entre quien cifra y quien descifra. La recuperación depende de usar los mismos alfabetos o la misma rejilla, parámetros y longitud, y el texto se recupera en forma normalizada cuando el preprocesado elimina información. Ambos métodos son apropiados para estudiar algoritmos clásicos, pero no para proteger datos reales.

Referencias

[1] Trinity College Department of Computer Science. «The Alberti Cipher,» visitado 21 de oct. de 2026. dirección: https://www.cs.trincoll.edu/~crypto/historical/alberti.html

[2] E. B. F. von Wostrowitz, *Handbuch der Kryptographie*. Wien: L. W. Seidel & Sohn, 1881. visitado 21 de oct. de 2026. dirección: https://books.google.com/books?id=Y7aa_IrgVrkC

[3] K. de Leeuw, «The Solving of a Fleissner Grille during an Exercise by the Royal Netherlands Army in 1913,» en *Proceedings of the 1st International Conference on Historical Cryptology*, 2018. dirección: https://ep.liu.se/en/conference-article.aspx?Article_No=12&issue=149&series=

[4] J. J. A. Cornejo. «Alberti Fleissner: implementación educativa de cifrados clásicos,» visitado 21 de oct. de 2026. dirección: https://github.com/ElQuesoHelado/Alberti_Fleissner